assignment4_part2

October 24, 2026

1 Assignment 4: Dealing with Sequences Real-World Data (Part II)

1.1 Edit Distance

In Part I of this assignment, we mainly focused on extracting patterns of sequence data. Now let's switch our attention to calculating the similarity/distance between sequences.

As described in the lecture, one effective measurement of distance is the Levenshtein Edit Distance, which calculates the smallest number of single-character edits (insertions, deletions, or substitutions) needed to transform one sequence to the other. Let's try this algorithm out in this assignment!

1.1.1 Exercise 3. Levenshtein Edit Distance (30 pts)

Please complete the `my_edit_distance` function to calculate the Levenshtein edit distance. Your function should "fill the table" as described in the lecture and return the filled table as a 2-d matrix. We have performed all necessary initialization.

Recall the algorithm for calculating Levenshtein edit distance:

2 Wagner-Fischer Algorithm

$$X = x_1x_2...x_n, \quad Y = y_1y_2...y_m$$

2.0.1 Recurrence Relations:

- **Base Cases:**

$$d(i, 0) = i, \quad d(0, j) = j$$

- **Recursive Case:**

$$d(i, j) = \min \begin{cases} d(i-1, j) + 1 \\ d(i, j-1) + 1 \\ d(i-1, j-1) + t(i, j) \end{cases}$$

- **Cost Function:**

$$t(i, j) = \begin{cases} 1, & \text{if } x_i \neq y_j \\ 0, & \text{if } x_i = y_j \end{cases}$$

[11]:
```
def my_edit_distance(string_x, string_y):
    # Left-pad a blank character to both strings
    string_x = ' ' + string_x
    string_y = ' ' + string_y

    # Obtain the length of the padded string
    len_x = len(string_x)
    len_y = len(string_y)

    # Initializing the distance matrix
    dist_mat = [[0] * len_y for i in range(len_x)]
    for i in range(len_x):
        dist_mat[i][0] = i
    for j in range (len_y):
        dist_mat[0][j] = j

    # Calculating the distance matrix row by row.
    for j in range(1, len_y):
        for i in range(1, len_x):
            # YOUR CODE HERE
            #raise NotImplementedError()
            dist_mat[i][j] = min(min(dist_mat[i-1][j]+1, dist_mat[i][j-1]+1),␣
 ↪dist_mat[i-1][j-1] + (string_x[i] != string_y[j]))
    return dist_mat
```

With this function, you can obtain the edit distance by visiting the bottom-right element of the table.

[12]:
```
dist_mat = my_edit_distance("VINTNER", "WRITERS")
print("edit distance = ", + dist_mat[-1][-1])
dist_mat
```

```
edit distance =  5
```

[12]:
```
[[0, 1, 2, 3, 4, 5, 6, 7],
 [1, 1, 2, 3, 4, 5, 6, 7],
 [2, 2, 2, 2, 3, 4, 5, 6],
 [3, 3, 3, 3, 3, 4, 5, 6],
 [4, 4, 4, 4, 3, 4, 5, 6],
 [5, 5, 5, 5, 4, 4, 5, 6],
 [6, 6, 6, 6, 5, 4, 5, 6],
 [7, 7, 6, 7, 6, 5, 4, 5]]
```

[13]:
```
# This code block test if the `my_edit_distance` is implemented correctly
# We hide some tests so passing the displayed assertions does not guarantee␣
 ↪full points.

assert my_edit_distance("VINTNER", "WRITERS") \
```

```
    == [[0, 1, 2, 3, 4, 5, 6, 7],
        [1, 1, 2, 3, 4, 5, 6, 7],
        [2, 2, 2, 2, 3, 4, 5, 6],
        [3, 3, 3, 3, 3, 4, 5, 6],
        [4, 4, 4, 4, 3, 4, 5, 6],
        [5, 5, 5, 5, 4, 4, 5, 6],
        [6, 6, 6, 6, 5, 4, 5, 6],
        [7, 7, 6, 7, 6, 5, 4, 5]]

assert my_edit_distance("birthday", "Birthdayyy") \
 == [[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
     [1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
     [2, 2, 1, 2, 3, 4, 5, 6, 7, 8, 9],
     [3, 3, 2, 1, 2, 3, 4, 5, 6, 7, 8],
     [4, 4, 3, 2, 1, 2, 3, 4, 5, 6, 7],
     [5, 5, 4, 3, 2, 1, 2, 3, 4, 5, 6],
     [6, 6, 5, 4, 3, 2, 1, 2, 3, 4, 5],
     [7, 7, 6, 5, 4, 3, 2, 1, 2, 3, 4],
     [8, 8, 7, 6, 5, 4, 3, 2, 1, 2, 3]]
```

In this assignment, we have implemented the edit distance by hand. In reality, however, you can find it in several packages. For example, `nltk` offers the `nltk.edit_distance` API along with several other distance metrics for sequences. You may check out its documentation here.